

5 · Physics-Informed Neural Networks (PINNs) ⇔

Instead of discretizing a PDE on a mesh, a **Physics-Informed Neural Network** trains a neural network $u_\theta(x)$ to satisfy the PDE's residual (and its boundary conditions) at randomly sampled collocation points. `NeuralPDE.jl` turns a symbolic PDE specification (written with `ModelingToolkit.jl`) directly into an `Optimization.jl` training problem.

We solve the 1D Poisson equation

$$u''(x) = f(x) = -\pi^2 \sin(\pi x), \quad u(0) = 0, u(1) = 0$$

whose exact solution is $u(x) = \sin(\pi x)$, so we can check the network's accuracy directly.

```
1 using NeuralPDE, Lux, ModelingToolkit, Optimization, OptimizationOptimisers, DomainSets
   , Random, Plots
```

1. Symbolic problem specification ⇔

`ModelingToolkit.jl` lets us write down the PDE almost exactly as it looks on paper.

► [x]

```
1 ModelingToolkit.@parameters x
```

⚠ Attempting to print a symbolic expression in a Pluto notebook. Please run ``import LaTeXify`` to enable pretty-printing of symbolic expressions. This warning will only display once.

► [u★]

```
1 ModelingToolkit.@variables u(..)
```

► [VarDomainPairing(x, 0.0 .. 1.0)]

```
1 begin
2     Dxx = ModelingToolkit.Differential(x)^2
3
4     f(x) = -π^2 * sin(π * x)
5     eq = Dxx(u(x)) ~ f(x)
6
7     bcs = [u(0.0) ~ 0.0, u(1.0) ~ 0.0]
8     domains = [x ∈ Interval(0.0, 1.0)]
9 end
```

2. The neural network ansatz ⇌

A small Lux network stands in for $u_\theta(x)$. NeuralPDE.jl handles differentiating it (via automatic differentiation) to build the residual loss automatically.

```
Chain(
  layer_1 = Dense(1 => 16, tanh),      # 32 parameters
  layer_2 = Dense(16 => 16, tanh),     # 272 parameters
  layer_3 = Dense(16 => 1),            # 17 parameters
)                                     # Total: 321 parameters,
                                   # plus 0 states.
```

```
1 begin
2   rng5 = Random.default_rng()
3   Random.seed!(rng5, 0)
4   chain = Chain(Dense(1, 16, tanh), Dense(16, 16, tanh), Dense(16, 1))
5 end
```

```
discretization =
```

```
► PhysicsInformedNN(Chain{@NamedTuple{layer_1::Dense{typeof(tanh)}, Int64, Int64, Nothing, Noth
```

```
1 discretization = PhysicsInformedNN(chain, QuadratureTraining())
```

```
OptimizationProblem. In-place: true
```

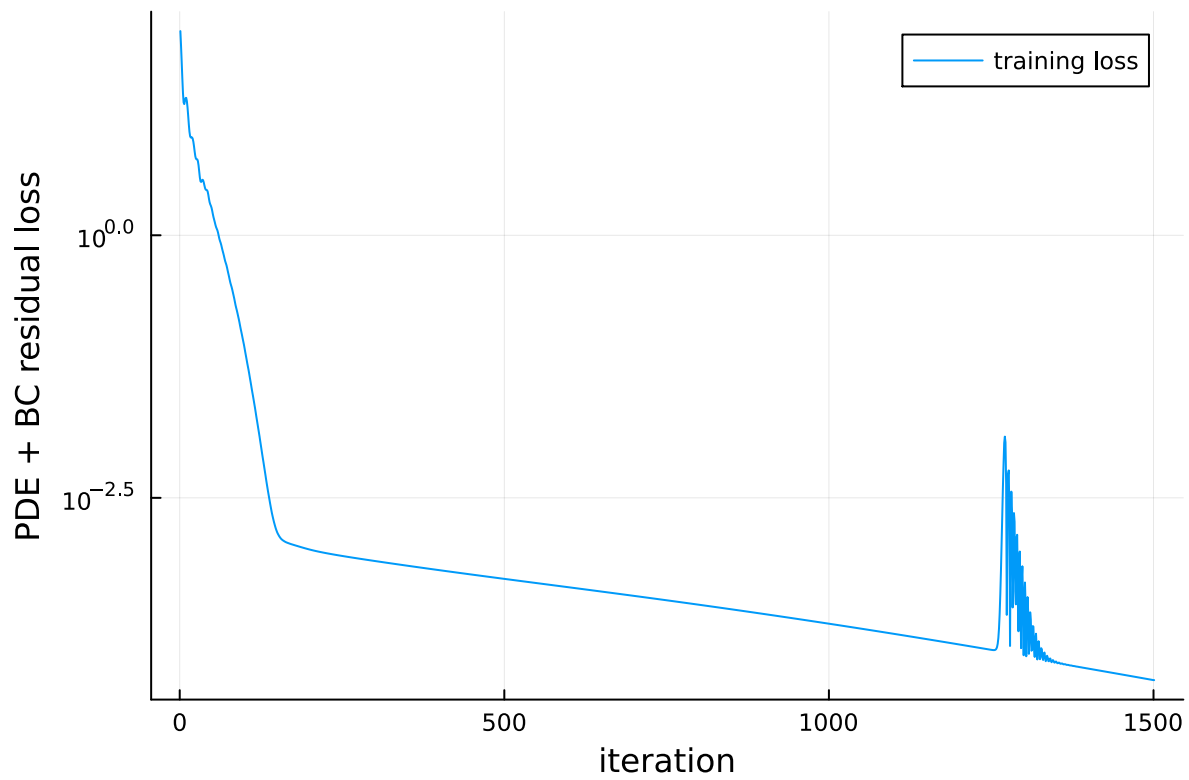
```
u0: ComponentVector{Float64}{layer_1 = (weight = [-0.3607475459575653; 0.00922261830419302; .
```

```
1 begin
2   @named pde_system = PDESystem(eq, bcs, domains, [x], [u(x)])
3   prob_pinn = discretize(pde_system, discretization)
4 end
```

3. Train the network to minimize the PDE + boundary-condition residual ⇌

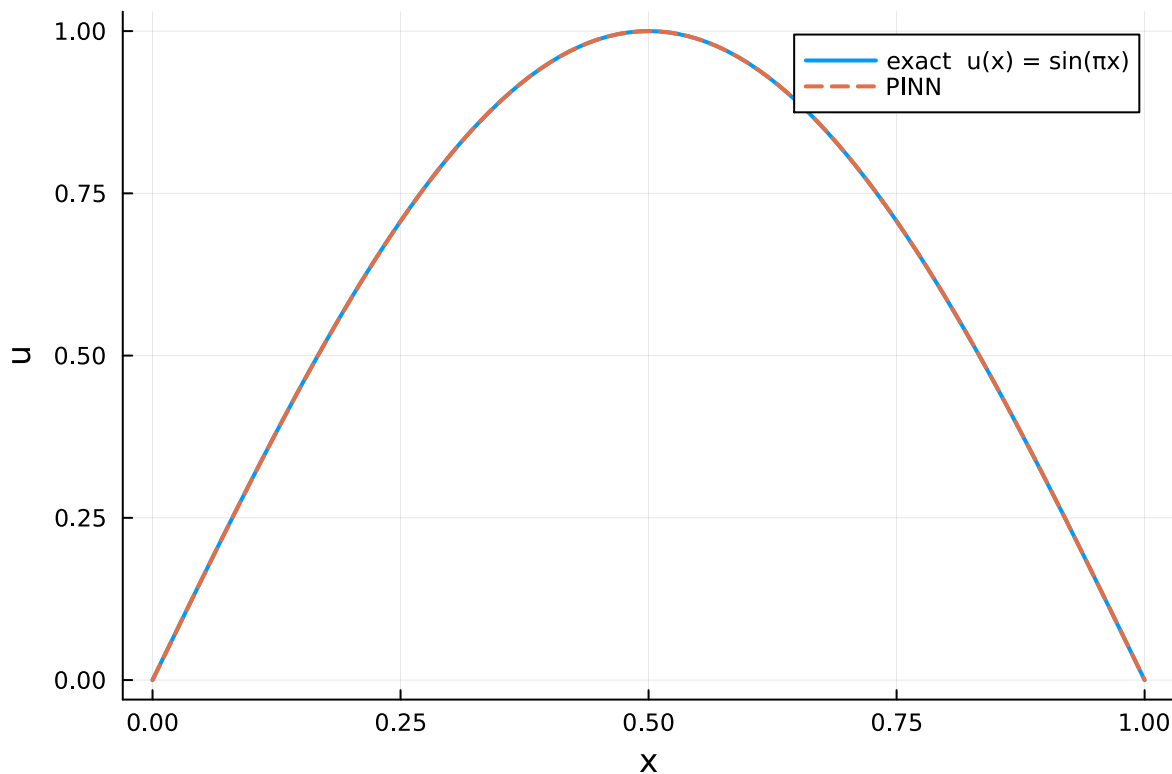
```
► SciMLBase.OptimizationSolution{Float64, 1, ComponentArrays.ComponentVector{Float64, Vector{Float6
```

```
1 begin
2   pinn_losses = Float64[]
3   pinn_callback = function (state, loss)
4     push!(pinn_losses, loss)
5     return false
6   end
7
8   res_pinn = Optimization.solve(prob_pinn, OptimizationOptimisers.Adam(0.01);
9     callback=pinn_callback, maxiters=1500)
9 end
```



```
1 plot(pinn_losses; xlabel="iteration", ylabel="PDE + BC residual loss", yscale=:log10,  
label="training loss")
```

4. Compare against the exact solution ⇄



```
1 begin
2   phi = discretization.phi
3   final_ps = res_pinn.u
4
5   xs = 0.0:0.01:1.0
6   u_predicted = [first(phi([xi], final_ps)) for xi in xs]
7   u_exact = sin.(π .* xs)
8
9   plot(xs, u_exact; label="exact u(x) = sin(πx)", lw=2, xlabel="x", ylabel="u")
10  plot!(xs, u_predicted; label="PINN", lw=2, ls=:dash)
11 end
```

0.00019947237989302835

```
1 maximum(abs.(u_predicted .- u_exact))
```

Takeaways ⇄

- A PINN needs no mesh: the loss is just "how well does the network satisfy the equation and boundary conditions," evaluated at sampled points.
- ModelingToolkit.jl symbolic expressions let NeuralPDE.jl build the residual and its derivatives automatically via automatic differentiation, rather than requiring you to hand-derive them.

- The same recipe scales to higher-dimensional PDEs and systems of PDEs where classical mesh-based methods become expensive — the main change is a bigger network and more training time, not new mathematics in the notebook.